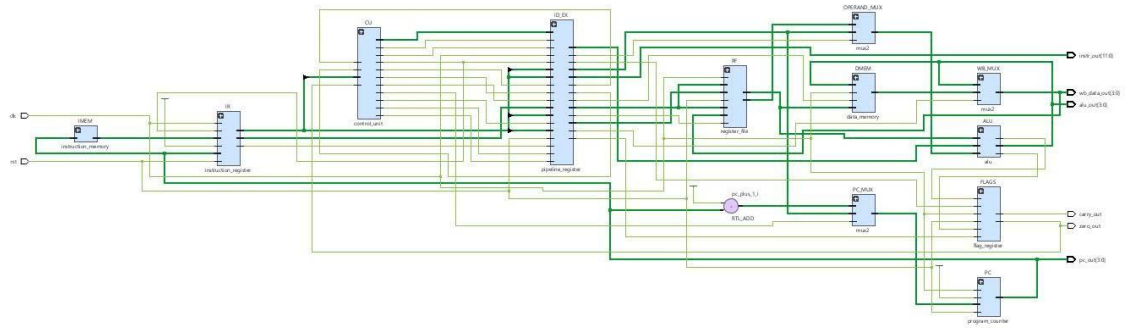
PROJECT REPORT

Design and Implementation of a

4-Bit RISC CPU

with a 3-Stage Pipeline Using Verilog HDL



RTL schematic of cpu_top (Vivado elaborated design)

Theory, Implementation, Simulation and Synthesis

4-bit datapath · 12-bit instructions · 4 registers · IF / ID / EX pipeline · hardwired control

Student	Bùi Hữu Lợi
Student ID	SE196720
University	FPT University
Major	IC Design
Tools	Xilinx Vivado 2026.1 (XSim), Verilog HDL
Target device	Artix-7 xc7a35tcpg236-1
Date	October 2026

1. Introduction

A CPU has two halves. The **datapath** (registers, ALU, memories, multiplexers and buses) stores and processes data. The **control path** decodes each instruction and drives the select, enable and write signals that make the datapath do the right thing.

This project designs, implements and verifies a small RISC-style 4-bit CPU in Verilog HDL. "4-bit" means that the registers, the ALU and the data memory all work on 4-bit numbers (0 to 15). The CPU uses a **3-stage pipeline**: while one instruction executes, the next is decoded and the one after that is fetched. Every block is a separate Verilog module, connected in `cpu_top`.

Objectives

1. Design a 4-bit ALU with ADD, SUB, AND, OR, XOR and NOT, and Zero and Carry flags.
2. Build a four-register file, a program counter, instruction and data memories, and the datapath multiplexers.
3. Design a hardwired control unit (instruction decoder and branch unit).
4. Organise the CPU as an IF / ID / EX pipeline and handle jumps correctly by flushing.
5. Integrate the modules, run test programs, and verify the design by simulation and synthesis in Vivado.

2. Specifications

Table 1. *Key specifications*

Parameter	Specification
Architecture	RISC, load/store, Harvard memory organisation
Datapath width	4 bits (registers, ALU operands, data memory)
Register file	R0-R3, 4 bits each, 2-bit register numbers; two read ports, one write port
Instruction word	12 bits: opcode 4, Rd 2, Rs 2, imm/addr 4
Instruction memory	16 words x 12 bits (ROM, loaded from a .mem file)
Data memory	16 words x 4 bits (RAM)
Program counter	4 bits (16 instruction addresses), wraps from 15 to 0
Instruction set	12 instructions; opcodes 1100-1111 are reserved and behave as NOP
Pipeline	3 stages: IF (fetch), ID (decode), EX (execute + memory + write-back)
Flags	Zero (result = 0000); Carry (carry-out for ADD, no-borrow for SUB)
Control	Hardwired: combinational decoder + branch unit
Implementation	Verilog-2005, Vivado 2026.1 (XSim), target Artix-7 xc7a35tcpg236-1

Table 2. *Parameters derived from the design*

Parameter	Value
Instruction / data memory size	192 bits (16 x 12) / 64 bits (16 x 4)
Unsigned / signed (two's complement) range	0 to 15 / -8 to +7
Latency of one instruction	3 clock cycles (IF, ID, EX)
Peak throughput	1 instruction per clock once the pipeline is full
Cycles for N instructions with J taken jumps	N + 2 + 2J (a taken jump costs 2 cycles; a not-taken JZ costs 0)
Estimated flip-flops (excluding memories)	about 75: PC 4, IF/ID 17, ID/EX 36, registers 16, flags 2

The flip-flop figure is estimated from the RTL; synthesis may remove trace-only fields in ID/EX. Actual resource usage must be read from the Vivado utilization report (Section 9).

3. Architecture and Pipeline

3.1 Load/store and Harvard organisation

All instructions are 12 bits long, so decoding is simple and pipelining is easy. Only LOAD and STORE access memory; the ALU never works directly on memory. The CPU uses a two-address format, Rd = Rd op Rs, so that Rd is both the first source and the destination, which leaves room for a 4-bit immediate in the 12-bit word.

Instructions and data are in separate memories. In the same clock cycle the IF stage reads an instruction while the EX stage may read or write data, so there is no structural hazard.

3.2 Three-stage pipeline

Each instruction passes through IF, ID and EX. With the pipeline full, one instruction finishes every clock (Figure 1). Two pipeline registers separate the stages: **IF/ID** (the instruction register) holds the fetched instruction, its address and a valid bit; **ID/EX** holds the decoded control signals, Rd, Rs, imm, the address and a valid bit.

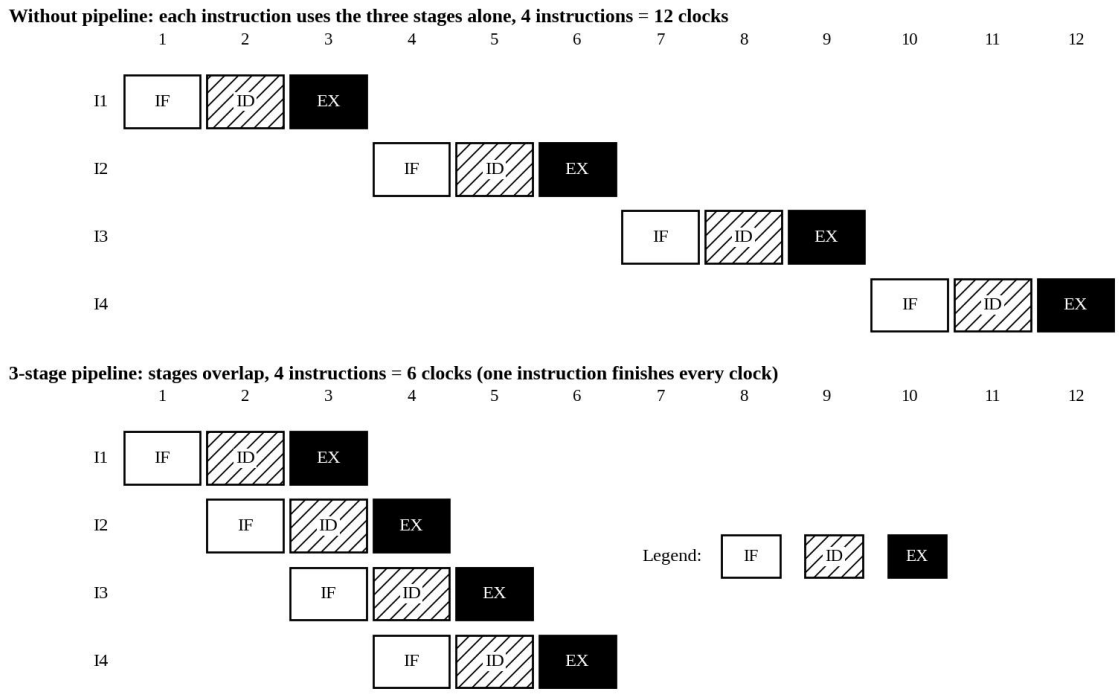


Figure 1. *Without a pipeline (top) four instructions need 12 clocks; with the 3-stage pipeline (bottom) they need 6.*

Table 3. *Work done in each pipeline stage*

Stage	Work	Blocks
IF	Read the instruction at address PC into IF/ID; compute PC + 1	PC, PC MUX, instruction memory, IF/ID
ID	Decode the opcode, generate control signals, store them in ID/EX	Decoder, ID/EX
EX	Read Rd/Rs, ALU, memory access, write back to Rd, update flags, decide jumps	Register file, operand MUX, ALU, flags, data memory, WB MUX, branch unit

3.3 Hazards

Table 4. *Pipeline hazards and how the CPU handles them*

Hazard	Risk	Solution
Structural	Two stages need the same hardware	Separate instruction and data memories (Harvard); every block belongs to one stage
Data	An instruction reads a register before the previous one has written it	Registers are read in EX, one clock after the previous instruction wrote them, so the new value is always seen. No forwarding or stall is needed
Control	After a jump, the instructions already fetched are wrong	The jump is decided in EX; the two instructions in IF/ID and ID/EX are flushed to NOPs (2 bubbles)

Example (data): `LDI R1, 5` followed by `ADD R1, R2`. LDI writes R1 at the end of its EX cycle; in the next cycle ADD is in EX and already reads 5. Flags behave the same way, so a JZ right after a SUB tests the new Zero flag.

Example (control): when a taken `JMP` is in EX, the branch unit sets `pc_sel = flush = 1`. At the clock edge the PC loads the jump address and both pipeline registers are emptied (Figure 2), costing 2 clock cycles.

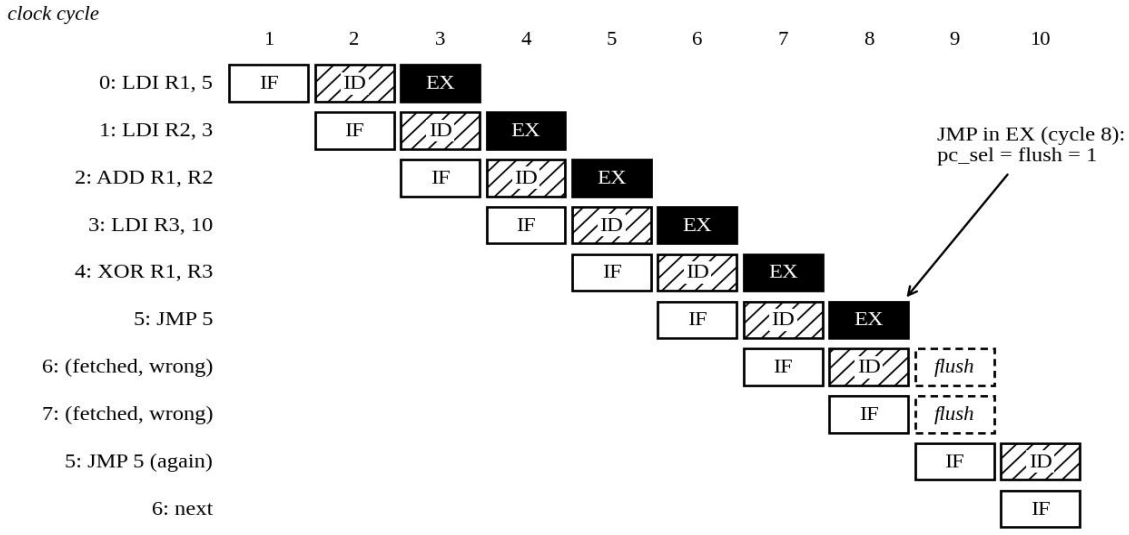


Figure 2. *Demo program: the taken JMP at address 5 flushes the two wrongly fetched instructions (addresses 6 and 7).*

4. Instruction Set Architecture

Every instruction uses the same 12-bit layout (Figure 3), so the decoder always finds each field in the same place. Register codes are R0 = 00, R1 = 01, R2 = 10, R3 = 11.

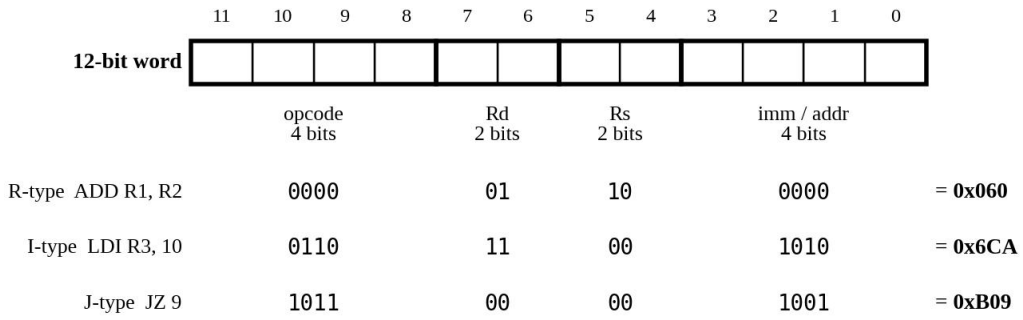


Figure 3. *12-bit instruction format with three encoding examples.*

Table 5. *Instruction set*

Opcode	Instruction	Operation	Flags	Example → hex
0000	ADD Rd, Rs	Rd ← Rd + Rs	Z, C	ADD R1, R2 → 060
0001	SUB Rd, Rs	Rd ← Rd - Rs	Z, C	SUB R1, R2 → 160
0010	AND Rd, Rs	Rd ← Rd AND Rs	Z	AND R3, R2 → 2E0
0011	OR Rd, Rs	Rd ← Rd OR Rs	Z	OR R3, R2 → 3E0
0100	XOR Rd, Rs	Rd ← Rd XOR Rs	Z	XOR R1, R3 → 470
0101	NOT Rd	Rd ← NOT Rd	Z	NOT R0 → 500
0110	LDI Rd, imm	Rd ← imm	-	LDI R1, 5 → 645
0111	MOV Rd, Rs	Rd ← Rs	-	MOV R2, R1 → 790
1000	LOAD Rd, addr	Rd ← DMEM[addr]	-	LOAD R0, 0 → 800
1001	STORE Rd, addr	DMEM[addr] ← Rd	-	STORE R1, 1 → 941
1010	JMP addr	PC ← addr	-	JMP 5 → A05
1011	JZ addr	if Zero = 1 then PC ← addr	-	JZ 9 → B09
1100-1111	(reserved)	NOP	-	C00

5. Datapath and Control Unit

Figure 4 shows the complete datapath divided into its three stages. Solid lines carry data; dashed lines are the control signals `pc_sel` and `flush` produced by the branch unit. The other control signals (`alu_ctrl`, `alu_src_sel`, `wb_sel`, `reg_we`, `mem_we`, `zero_we`, `carry_we`, `is_jmp`, `is_jz`) are generated by the decoder in ID and travel with the instruction in the ID/EX register.

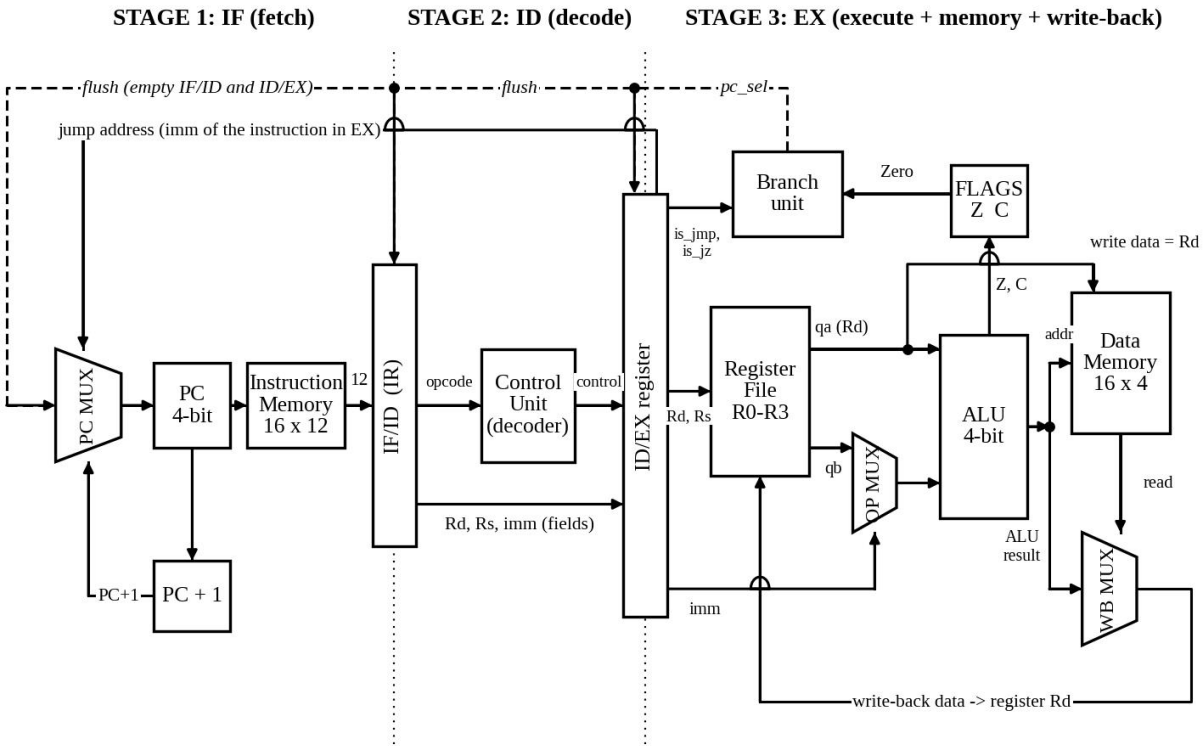


Figure 4. *Datapath of the 3-stage pipelined 4-bit RISC CPU.*

5.1 ALU and flags

The ALU is purely combinational: a shared adder/subtractor (ADD, SUB), a logic unit (AND, OR, XOR, NOT, PASS), a result multiplexer controlled by `alu_ctrl`, and a zero detector. Subtraction uses A - B = A + ~B + 1: B passes through XOR gates controlled by `do_sub`, which is also the carry-in. PASS copies operand B to the output and is used by LDI, MOV, LOAD and STORE (the immediate or address travels through the ALU).

Table 6. *ALU control codes*

alu_ctrl	Operation	Result	Used by
0	ADD	a + b	ADD
1	SUB	a - b	SUB
2	AND	a & b	AND
3	OR	a | b	OR
4	XOR	a ^ b	XOR
5	NOT	~a	NOT
6	PASS	b	LDI, MOV, LOAD, STORE

Carry convention (identical in the ALU, flag register and testbench): for ADD, Carry = 1 means the sum did not fit in 4 bits; for SUB, Carry = 1 means no borrow (A ≥ B) and Carry = 0 means a borrow occurred, as in ARM. Examples: 5 + 3 = 1000 (Z = 0, C = 0); 15 + 1 = 0000 (Z = 1, C = 1); 5 - 3 = 0010 (Z = 0, C = 1); 3 - 5 = 1110 (Z = 0, C = 0); 7 - 7 = 0000 (Z = 1, C = 1).

5.2 Instruction decoder (ID stage)

The decoder is combinational and written with safe defaults: every signal is first set to its "do nothing" value and the `case` statement turns on only what the opcode needs. This gives a known output for every opcode, avoids latches, and makes the reserved opcodes behave as NOP.

Table 7. *Decoder outputs for every opcode*

Instruction	alu_ctrl	alu_src	wb_sel	reg_we	mem_we	zero_we	carry_we	is_jmp	is_jz
ADD	ADD	0	0	1	0	1	1	0	0
SUB	SUB	0	0	1	0	1	1	0	0
AND/OR/XOR/NOT	op	0	0	1	0	1	0	0	0
LDI	PASS	1	0	1	0	0	0	0	0
MOV	PASS	0	0	1	0	0	0	0	0
LOAD	PASS	1	1	1	0	0	0	0	0
STORE	PASS	1	0	0	1	0	0	0	0
JMP	-	0	0	0	0	0	0	1	0
JZ	-	0	0	0	0	0	0	0	1
Reserved	-	0	0	0	0	0	0	0	0

5.3 Branch unit (EX stage)

Table 8. *Branch unit truth table*

Instruction in EX	Zero	pc_sel	flush	Result at the clock edge
Not a jump	any	0	0	PC ← PC + 1; pipeline advances normally
JMP	any	1	1	PC ← jump address; IF/ID and ID/EX emptied
JZ	0	0	0	Not taken; continue normally
JZ	1	1	1	PC ← jump address; IF/ID and ID/EX emptied

A conditional jump is just a multiplexer: for JZ, `pc_sel` equals the stored Zero flag.

6. RTL Implementation

The design is split into one module per block; `cpu_top` only connects them, stage by stage, with signal names starting with `id_` or `ex_`. Sub-modules are written in the same file as their parent: `alu.v` contains `adder_subtractor`, `logic_unit` and `zero_detector`, and `control_unit.v` contains `instruction_decoder` and `branch_unit`. Table 9 lists the instances of `cpu_top`, using the names shown in the RTL schematic (Figure 5).

Figure 5 is the RTL schematic generated by Vivado after elaboration of `cpu_top`. It contains all designed blocks (IMEM, IR, CU, ID_EX, RF, OPERAND_MUX, ALU, FLAGS, DMEM, WB_MUX, PC_MUX, PC) and the PC + 1 adder, with the observation outputs `pc_out[3:0]`, `instr_out[11:0]`, `alu_out[3:0]`, `wb_data_out[3:0]`, `zero_out` and `carry_out`.

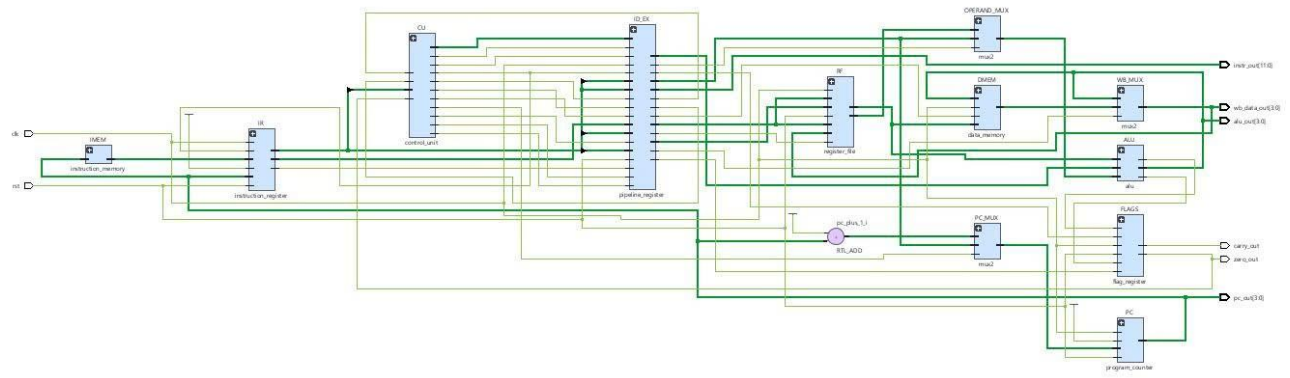


Figure 5. *RTL schematic of* `cpu_top` *after elaboration (Vivado 2026.1).*

Table 9. *Main functional blocks of the datapath (instance names as in the RTL schematic)*

Instance	Module (RTL)	Role
PC	`program_counter`	Stores the current instruction address (4 bits)
RTL_ADD	(adder)	Computes PC + 1
PC_MUX	`mux2`	Selects PC + 1 or the branch address (based on `pc_sel`)
IMEM	`instruction_memory`	Program memory, combinational read based on the address
IR	`instruction_register`	IF/ID pipeline register, with `flush` signal and `valid` bit
CU	`control_unit`	Decodes the opcode and makes branch decisions (`pc_sel`, `flush`)
ID_EX	`pipeline_register`	ID/EX pipeline register, stores rd, rs, imm and control signals
RF	`register_file`	4 registers x 4 bits, 2 read ports (A, B), 1 write port
OPERAND_MUX	`mux2`	Selects ALU operand B: register or `imm` constant (`alu_src_sel`)
ALU	`alu`	Performs ADD, SUB, AND, OR, XOR, NOT, PASS B; outputs `carry`, `zero`
FLAGS	`flag_register`	Stores Zero/Carry flags according to `zero_we`, `carry_we`
DMEM	`data_memory`	16 x 4-bit data memory for LOAD/STORE
WB_MUX	`mux2`	Selects write-back data: ALU result or DMEM data (`wb_sel`)

Key code: the decoder (excerpt) and the branch unit.

```
always @(*) begin
  alu_ctrl = ALU_ADD; alu_src_sel = 1'b0; wb_sel = 1'b0;      // defaults: do nothing
  reg_we = 1'b0; mem_we = 1'b0; zero_we = 1'b0; carry_we = 1'b0;
  is_jmp = 1'b0; is_jz = 1'b0;
  case (opcode)
    ADD:   begin alu_ctrl = ALU_ADD; reg_we = 1'b1; zero_we = 1'b1; carry_we = 1'b1; end
    LDI:   begin alu_ctrl = ALU_PASS; alu_src_sel = 1'b1; reg_we = 1'b1; end
    LOAD:  begin alu_ctrl = ALU_PASS; alu_src_sel = 1'b1; wb_sel = 1'b1; reg_we = 1'b1; end
    STORE: begin alu_ctrl = ALU_PASS; alu_src_sel = 1'b1; mem_we = 1'b1; end
    JMP:   is_jmp = 1'b1;
    JZ:    is_jz  = 1'b1;
    default: ;                                          // reserved opcodes = NOP
  endcase
end

always @(*) begin                                       // branch unit (EX stage)
  if (is_jmp)                  pc_sel = 1'b1;       // JMP always jumps
  else if (is_jz && zero_flag) pc_sel = 1'b1;       // JZ jumps when Zero = 1
  else                         pc_sel = 1'b0;
end
assign flush = pc_sel;                                  // taken jump -> flush
```

7. Test Programs

Programs are written as hexadecimal machine code in `.mem` files, one 3-digit word per line starting at address 0; unused words are C00 (NOP). Four programs are used: `program.mem` (demo), `prog_arith.mem` (ADD, SUB, STORE, LOAD, Zero), `prog_logic.mem` (AND, OR, XOR, MOV, NOT, LDI) and `prog_jump.mem` (PC increment, JMP, taken and not-taken JZ, flushing).

Table 10. *Demonstration program (program.mem)*

Addr	Hex	Instruction	Result
0	645	LDI R1, 0101	R1 = 0101 (5)
1	683	LDI R2, 0011	R2 = 0011 (3)
2	060	ADD R1, R2	R1 = 1000 (8)
3	6CA	LDI R3, 1010	R3 = 1010
4	470	XOR R1, R3	R1 = 0010 (2)
5	A05	JMP 5	Loop in place for a stable demonstration

Table 11. *Jump test program (prog_jump.mem)*

Addr	Hex	Instruction	Effect
0	601	LDI R0, 1	R0 = 1; PC 0 → 1 (increment)
1	A04	JMP 4	PC = 0100; addresses 2 and 3 are flushed
2-3	60F	LDI R0, 15	must never run
4-5	647, 150	LDI R1, 7; SUB R1, R1	R1 = 0000, Zero = 1
6	B09	JZ 9	taken: PC = 9; addresses 7 and 8 are flushed
9-10	6C2, 0F0	LDI R3, 2; ADD R3, R3	R3 = 0100, Zero = 0
11	B0E	JZ 14	not taken: continue at address 12
12-13	686, A0D	LDI R2, 6; JMP 13	R2 = 0110; stop

8. Verification and Simulation Results

8.1 Testbench

Verification uses the self-checking testbench `tb_cpu.v` running on the Vivado simulator (XSim). It loads each `.mem` program into the instruction memory, resets the CPU, waits until the instruction under test is in EX, compares the result with the expected value and prints PASS or FAIL, then a final summary. Helper tasks: `load_program` (load a file and clear data memory), `reset_cpu` (hold reset for 2 clocks), `finish(addr)` (wait until the instruction at `addr` has completed; waiting by instruction is used because the cycle count is not fixed after jumps) and `check` (compare a 4-bit value and count errors). Figure 6 shows the end of the testbench: Test 3 (logic), Test 4 (jumps) and the summary that calls `$finish` at line 123.



Figure 6. *End of* `tb_cpu.v`*: Test 3, Test 4 and the final summary.*

8.2 Results

Figure 7 shows the Tcl console after simulation: all 7 checks pass and the testbench reports `tb_cpu PASSED all 7 checks!`; `$finish` is called at 326 ns.



Figure 7. *Simulation output in the Vivado Tcl console.*

Table 12. *Checks performed by tb_cpu (7 / 7 passed, errors = 0)*

#	Program	Check	Value	Result
1	program.mem	ADD R1, R2 (5 + 3)	1000	PASS
2	program.mem	XOR R1, R3	0010	PASS
3	prog_arith.mem	ADD R1, R2 → R1	1000	PASS
4	prog_arith.mem	STORE R1, 0 → data memory [0]	1000	PASS
5	prog_arith.mem	SUB R1, R2 → R1	0010	PASS
6	prog_logic.mem	AND → R3	1000	PASS
7	prog_jump.mem	JMP 4: PC becomes 0100	0100	PASS

8.3 Waveforms

Figure 8 shows the whole simulation from 0 to 326 ns. `rst` is pulsed before each test program, `checks` counts up to 7, `errors` stays at 0, and `pc_out` and `alu_out` follow the program being executed. Figure 9 is the end of the run at 326 ns, where the Objects window shows `errors = 0`, `checks = 7`, `pc_out = 4`, `instr_out = c00` (a NOP in EX) and both flags at 0.

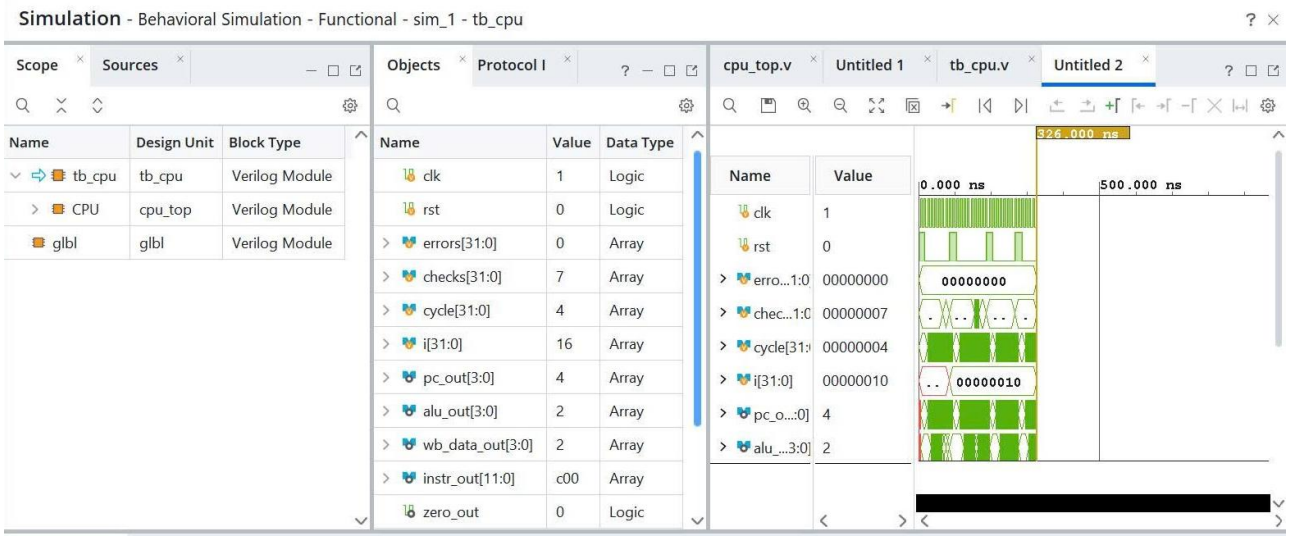


Figure 8. *Waveform of `tb_cpu` over 326 ns (clk, rst, errors, checks, cycle, i, pc_out, alu_out).*

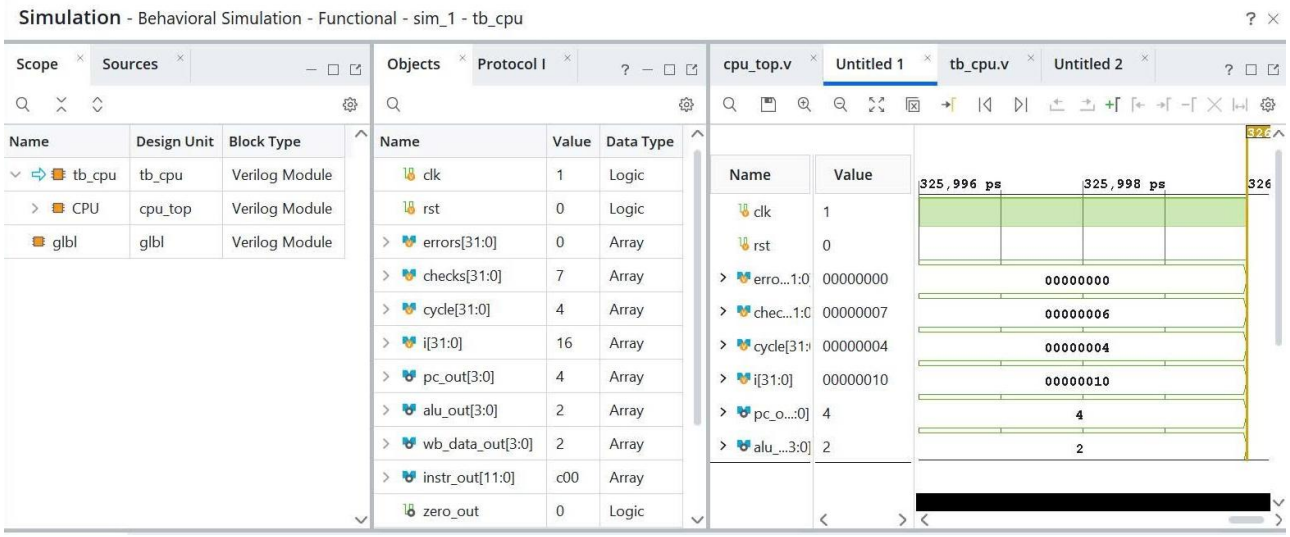


Figure 9. *Signal values at the end of the simulation (326 ns).*

9. Synthesis and Timing

The design was synthesised in Vivado for the Artix-7 **xc7a35tcpg236-1**, and Vivado reports **Synthesis Complete** (Figure 10). The netlist tree of `cpu_top` shows 37 nets and 31 leaf cells at the top level, plus hierarchical cells such as `PC` (`program_counter`).

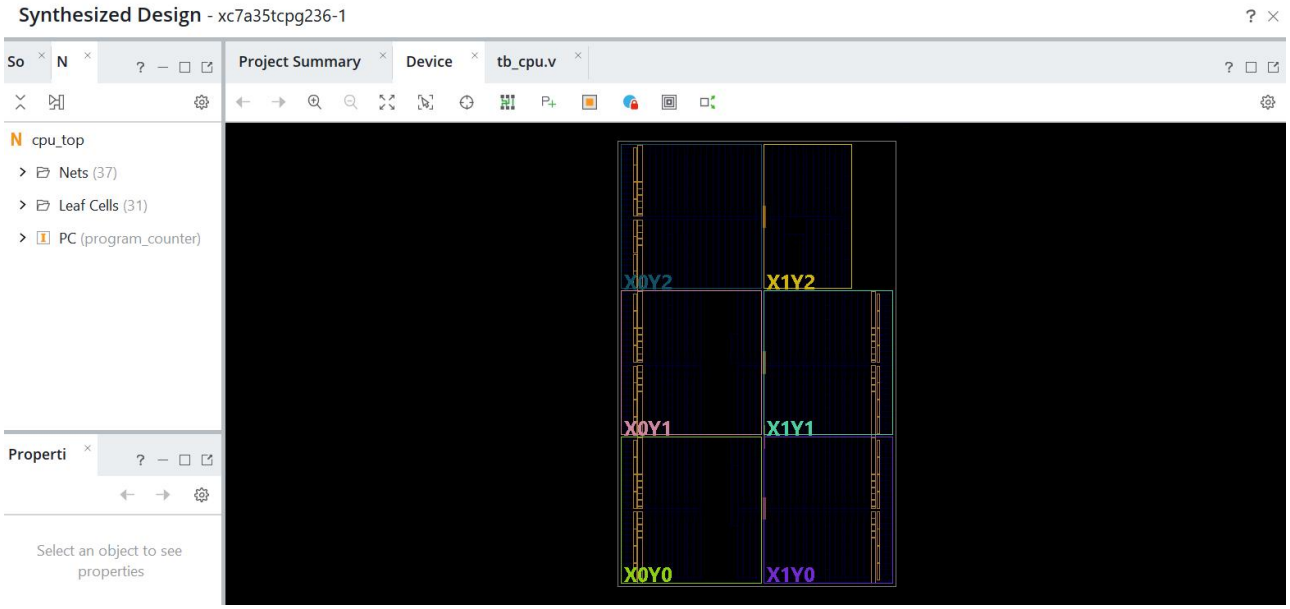


Figure 10. *Synthesized design of `cpu_top` on xc7a35tcpg236-1.*

Figure 11 shows the Check Timing report of run `impl_1`: 17 high-severity warnings, all caused by missing constraints (8 unconstrained internal endpoints, 4 with no clock, 4 without output delay, 1 without input delay).

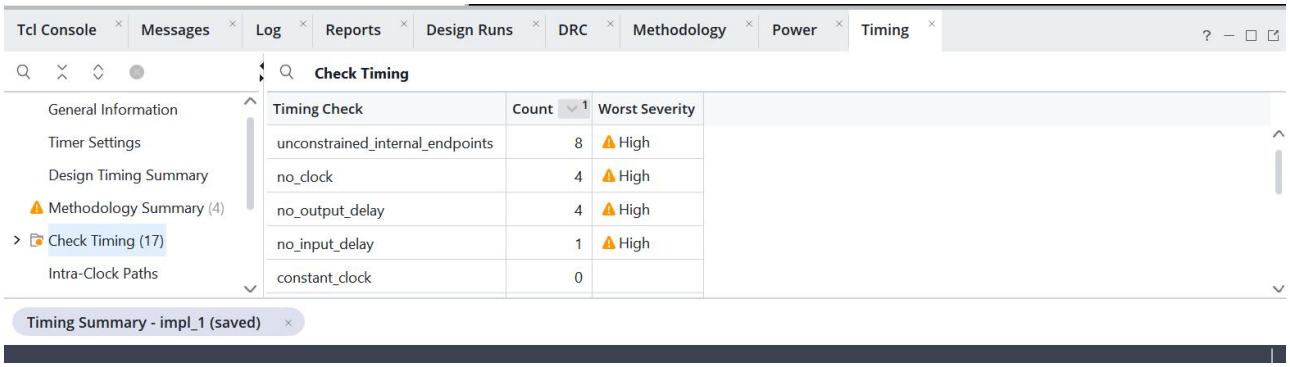


Figure 11. *Check Timing report in Vivado: 17 missing-constraint warnings.*

Because no clock constraint has been defined yet, WNS and the maximum clock frequency are not meaningful, so no operating frequency is claimed in this report. To obtain real timing and resource numbers, add a constraint file and re-run implementation, then run the utilization report. Example constraints for a 100 MHz target:

```
# constraints.xdc
create_clock -period 10.000 -name clk [get_ports clk]
set_input_delay  -clock clk 2.000 [get_ports rst]
set_output_delay -clock clk 2.000 [get_ports {pc_out[*] instr_out[*] alu_out[*] wb_data_out[*]
zero_out carry_out}]
```

10. Performance

Cycle counts follow from the pipeline model: N instructions without jumps need N + 2 cycles, and each taken jump adds 2. CPI is cycles divided by instructions. The last column compares with a non-pipelined design that needs 2 clocks per instruction.

Table 13. *Clock cycles (calculated with N + 2 + 2J)*

Program	Instr.	Cycles	CPI	2 clk/instr	Speed-up
Demo (up to XOR)	5	7	1.40	10	1.4x
Arithmetic and memory	9	11	1.22	18	1.6x
Logic	12	14	1.17	24	1.7x
Jumps (2 taken jumps)	10	16	1.60	20	1.25x
Long program, no jumps	N	N + 2	→ 1	2N	close to 2x

11. Limitations and Conclusion

Limitations and possible extensions

- A taken jump costs 2 cycles because it is decided in EX; deciding JMP in ID or adding branch prediction would reduce this.
- Programs are limited to 16 instructions and 16 data words by the 4-bit PC and address fields; wider fields, relative jumps or register-indirect LOAD/STORE would remove the limits.
- Only one conditional jump exists; the reserved opcodes can hold JC, JNZ, shifts or HALT. Signed overflow and negative flags, and CALL/RET, are not implemented.
- Timing constraints are not yet defined, so the maximum clock frequency and resource usage still have to be measured.

Conclusion. The project designed, implemented and verified a 4-bit RISC CPU in Verilog HDL. The 12-bit instruction format and the 12-instruction set were fixed first; the datapath (4-bit ALU, two-port register file, PC, 16 x 12 instruction memory, 16 x 4 data memory and three multiplexers) is driven by a hardwired control unit and organised as a 3-stage pipeline with IF/ID and ID/EX registers. Once the pipeline is full, one instruction finishes every clock; data hazards are avoided by reading registers in EX, and control hazards are handled by flushing two instructions after each taken jump. The integrated testbench ran 7 checks, all passing with zero errors in 326 ns of simulation, and the design synthesised successfully for the Artix-7 xc7a35tcpg236-1. The remaining step is to add clock constraints to determine the operating frequency and to record resource utilization.